



Lab 1 · Your First Agent-Assisted Task

Writing a CLAUDE.md and shipping a real PR with Plan Mode



What you'll do this week

Author a repo CLAUDE.md, then ship one real, mergeable AI-assisted PR using Plan Mode — end to end.

The flow

1. Review the Plan & talk about lab time time (do we need to change it)
2. Author your CLAUDE.md (stack, conventions, key dirs, testing, deploy)
3. Smoke-test it — Claude answers 3 repo questions from context alone

Lab 1.b (next week)

4. Pick a small backlog task; enter Plan Mode; review the plan first
5. Execute with Claude — course-correct, don't accept blindly
6. Write tests — ask for edge cases and error paths
7. Open a PR with a clear description; get a human reviewer

DONE WHEN

- CLAUDE.md committed + passes the smoke test
- A mergeable PR
- Tests written
- ≥ 1 human reviewer engaged

Counts toward Agentic Developer cert.



Writing your CLAUDE.md

The single habit the rest of the program builds on.



What it is, and why it matters

- Claude's durable memory for your project — loaded into context automatically at the start of every session.
- A single source of truth for how your repo works, so you don't re-explain it each time.
- Lives in the repo, committed and shared — the whole team (and Claude) works from the same context.
- The foundation for everything later: sharper plans, fewer wrong turns, less "yes-man" drift.

WHY IT MATTERS

Better context in → better code out. A shared CLAUDE.md turns siloed, personal setups into common ground — the shift from individual AI use to a team standard.



What goes in it — the five essentials

- | | | |
|---|-----------------------------|---|
| 1 | Stack & overview | Languages, frameworks, databases; what the project does in a sentence or two. |
| 2 | Conventions | Code style, naming, error handling, logging — the patterns to match. |
| 3 | Key directories | Where things live: entrypoints, domain logic, tests, migrations. |
| 4 | Testing approach | How tests are structured and the exact command to run them. |
| 5 | Deployment | How code ships — build/CI, environments, and any gotchas. |

Aim for ~100–200 lines. Never include secrets, tokens, or credentials.



The FSL template — start here

Project – <name>

<what it does – 1-2 lines>

Stack & overview

- Language / runtime: <...>
- Frameworks: <...>
- Database: <...>

Commands

- Build: <cmd> Test: <cmd>
- Lint: <cmd> Run: <cmd>

Code conventions

- <error-handling rule>
- <naming / structure rule>
- <logging rule>

Architecture & key dirs

- <dir> - <purpose>
- <dir> - <purpose>
- <dir> - <purpose>

Testing approach

- <how tests are structured>
- Run <test cmd> before every PR

Deployment

- <build / CI / environments>
- Gotchas: <known issues>

Do / Don't

- Do: <...>
- Don't: <...> · never commit secrets

Copy this, fill the <placeholders>, and delete what doesn't apply — this is your starting point.



Write it so Claude actually uses it

- Be specific, not generic — "we use X because Y," never "write clean code."
- Keep it short — 100–200 lines; if it sprawls, split into nested files.
- Use headings and bullets — scannable beats prose.
- Give exact, copy-pasteable commands — build, test, lint, run.
- Point to key files and directories instead of pasting them.
- Document the why behind a convention, not just the what.
- Treat it as living — update it the moment a pattern changes.
- Never commit secrets, tokens, or credentials.



Common pitfalls to avoid

Too long / bloated

Claude skims — the signal gets lost in the noise.

Vague filler

"Write good code" wastes context and changes nothing.

Stale content

Worse than none — it actively misleads the agent.

Secrets or internal URLs

It's committed to the repo. Keep them out.

Duplicating the README

Link to it; don't paste it in wholesale.

Dumping whole style guides

Summarize the rules and point to the source.



Lazy-load the README — point, don't import

POINTER = LAZY (recommended)

Loads on demand, when Claude reads it

Docs — read on demand

- Setup/run -> read README.md
- Arch -> read docs/arch.md first

~Zero context cost until used. Best for large or occasional docs.

@IMPORT = EAGER

Loads the full file every session at startup

@README.md

(full README in context every time)

Only for short, always-needed content — not a big README.

For a README, keep the pointer (lazy) and give a clear trigger — "before X, read Y" — so Claude fetches it at the right moment. There's no special lazy-import syntax; it rides on Claude reading files on demand.



Draft it in minutes, then refine

1. Run `/init` — Claude scans the repo and drafts a starter `CLAUDE.md`.
2. Edit it down — keep what's true and useful; start from the FSL template.
3. Fix what it missed — add the real commands and conventions.
4. Capture as you go — say 'Add to `CLAUDE.md`: `<rule>`' and Claude edits it for you (use `/memory` to review).
5. Commit it — it's now shared team context.

HANDY TO KNOW

- Personal overrides → `CLAUDE.local.md` (git-ignored).
- Pull in other docs with `@path` imports.
- Big repo? Add nested `CLAUDE.md` files per major directory.



Smoke-test before you rely on it

Start a fresh session and ask Claude three questions it can only answer from your CLAUDE.md:

? "How do I run the tests?"

? "Where does <feature> live, and what pattern does it follow?"

? "How do we deploy?"

If it can't answer, the gap
is in your CLAUDE.md.
Add it and retry.

Passing this 3-question smoke test is part of Lab 1 acceptance.



A short, real-world CLAUDE.md

Project – Booking API

Go service that books equipment reservations. Legacy .NET being modernized to Go.

Commands

Build: make build Test: make test Lint: golangci-lint run

Conventions

- Errors: wrap with %w, never swallow; log via internal/log.
- SQL: parameterized queries only; schema changes in /migrations.

Key directories

/cmd entrypoints · /internal domain logic · /migrations schema

Testing

Table-driven tests; cover error paths. Run make test before every PR.

Deployment

CI builds on merge to main; deploy via the runbook (link in /docs).



Your CLAUDE.md checklist

- ✓ Covers all five essentials (stack, conventions, key dirs, testing, deploy)
- ✓ 100–200 lines, scannable — headings + bullets
- ✓ Exact build / test / lint / run commands included
- ✓ No secrets, tokens, or credentials
- ✓ Passes the 3-question smoke test
- ✓ Committed to the repo

THEN

Pick a small backlog task and ship it: Plan Mode → tests → PR → human review. Your CLAUDE.md makes every step sharper.



Now build yours

Questions & support: post in your team's AI-training Slack channel — your instructor and the Applied AI Engineer are there.